Middle East Technical University | Department of Computer Engineering

CENG 499

Introduction to Machine Learning

Spring 2025-2026

Homework 4

Instance-Based Learning

Regulations

1. The homework is due by **23:55 on May 3rd, 2026**. Late submission is not allowed.
2. Submissions are to be made via ODTUClass; do not send your homework via e-mail.
3. **This is an individual homework.** You may discuss general ideas with your classmates, but the code and the report you submit must be entirely your own work.
4. As the syllabus states, the use of generative AI is forbidden and you are expected to obey this rule. In particular, **AI-generated content in reports will be punished strictly according to university regulations**. We do not want to see **any AI-generated content in the report**; all explanations must be written entirely by you. **This does not mean that the use of AI tools is allowed for the coding parts.** We reserve the right to conduct **oral examinations** on homework submissions if necessary.
5. Your code must run with **Python 3**.
6. Use **random_state = 42** for data splitting (and any other randomness), so that results are reproducible.
7. You may use libraries for basic data operations and for implementing data structures such as priority queues. However, **the learning algorithms (kNN, distance-weighted kNN, and fuzzy kNN) must be implemented entirely by yourself, from scratch**.
8. Write clean and modular code.
9. Submit a single zip file named `<your_student_id>.zip`.
   The zip file must include:
   - `<your_student_id>.pdf` (your report).
   - `<your_student_id>.py` (your script).
10. Your **report must be typeset and searchable** (e.g., using LaTeX or a word processor). Handwritten submissions are not accepted.
11. Your script should **run from the command line without modification**.
12. Send an e-mail to **garipler@metu.edu.tr** if you need to get in contact.

Question 1 (20 pts) - Fuzzy kNN

In this question, you will manually apply the **fuzzy kNN** algorithm *(check Q2 for the details of the algorithm)* on a small dataset. Write your answers using a typesetting tool such as LaTeX or a word processor.

We will work in $\mathbb{R}^3$ with the label set:

$$\{C_1, C_2, C_3, C_4\}$$

You are given the following training instances with their class membership distributions:

- $x_1 = (1, 1, 1), \quad \mu(x_1) = (0.7, 0.2, 0.1, 0.0)$
- $x_2 = (2, 1, 0), \quad \mu(x_2) = (0.1, 0.6, 0.2, 0.1)$
- $x_3 = (0, 2, 2), \quad \mu(x_3) = (0.2, 0.2, 0.5, 0.1)$

You are also given a test instance:

$$x_* = (1, 1, 0)$$

Use **fuzzy kNN** with:

$$k = 3, \quad m = 2, \quad \varepsilon = 10^{-8}$$

The membership of x_* to class C_j is defined as:

$$u_j(x_*) = \frac{\sum_{i=1}^{k} \mu_j(x_i) \left(\frac{1}{d(x_*, x_i) + \varepsilon} \right)^{\frac{2}{m-1}}}{\sum_{i=1}^{k} \left(\frac{1}{d(x_*, x_i) + \varepsilon} \right)^{\frac{2}{m-1}}}$$

where $d(x_*, x_i)$ is the Euclidean distance.
For the given test instance find the predicted class by following those steps:

a. Compute the distances from x_* to each training point.

b. Compute the weights for each neighbor. Note that for $m = 2$, the weights are given by:

$$w_i = \left(\frac{1}{d(x_*, x_i) + \varepsilon} \right)^2$$

c. Compute the membership vector:

$$(u_1(x_*), u_2(x_*), u_3(x_*), u_4(x_*))$$

d. Verify that the membership values sum to 1.

e. What is the predicted class of x_*? Is there anything unusual? Comment. (Hint: compare the membership values.)

Question 2 (80 pts) - kNN and Extensions from Scratch

In this question, you will study the **k-Nearest Neighbors (kNN)** algorithm and its extensions, **distance-weighted kNN** and **fuzzy kNN**, on a tabular dataset.

You may use libraries for basic data operations and data structures (e.g., priority queues). However, **you must implement the learning algorithms (kNN, distance-weighted kNN, and fuzzy kNN) from scratch**. Using library implementations of these algorithms is not allowed.

The classical kNN algorithm classifies a test instance by finding its k nearest neighbors in the training set (according to a distance metric such as Euclidean distance), and assigning the most frequent class among those neighbors.

A common extension is **distance-weighted kNN**, where each neighbor contributes with a weight inversely proportional to its distance. In this case, closer neighbors have a stronger influence on the prediction.

A further extension is **fuzzy kNN**. Instead of assigning a single class label, fuzzy kNN computes a **membership value** for each class. These values indicate the degree to which the test instance belongs to each class.

Fuzzy kNN. Let x be a test instance, and let x_i denote its k nearest neighbors. The membership of x to class c_j is defined as:

$$u_j(x) = \frac{\sum_{i=1}^{k} u_j(x_i) \left(\frac{1}{d(x,x_i)+\varepsilon}\right)^{\frac{2}{m-1}}}{\sum_{i=1}^{k} \left(\frac{1}{d(x,x_i)+\varepsilon}\right)^{\frac{2}{m-1}}}$$

where:

- $d(x, x_i)$ is the Euclidean distance between x and its i-th neighbor,
- $m > 1$ is the **fuzzifier** parameter,
- ε is a small constant to avoid division by zero,
- $u_j(x_i)$ is the membership of neighbor x_i to class c_j.

In this homework, the training instances are assumed to have **crisp labels** (i.e. each training instance belongs to exactly one class). Therefore, their memberships are defined as:

$$u_j(x_i) = \begin{cases} 1, & \text{if } x_i \text{ belongs to class } c_j \\ 0, & \text{otherwise} \end{cases}$$

Thus, fuzzy kNN produces a membership vector:

$$(u_1(x), u_2(x), \ldots, u_C(x))$$

and the predicted class is the one with the highest membership value.

Task Definition. You are given the **Glass Identification** dataset:

`https://archive.ics.uci.edu/dataset/42/glass+identification`

Implement the following methods:

- Classical kNN
- Distance-weighted kNN with weights
$$w_i = \frac{1}{d_i^2 + \varepsilon}$$
- Fuzzy kNN using the previously given formula.

Use Euclidean distance in all methods.

In both distance-weighted kNN and fuzzy kNN, use:

$$\varepsilon = 10^{-8}$$

In case of a tie, select the class with the smallest index.

For the experiments, use the following parameter values:

$$k \in \{4, 8\}, \quad m \in \{1.5, 2.5\}$$

Perform the following:

i. For classical kNN and distance-weighted kNN, evaluate both values of k.

ii. For fuzzy kNN, evaluate all combinations of k and m.

iii. For all methods and parameter settings, report:

- accuracy,
- macro-averaged precision,
- macro-averaged recall,
- macro-averaged F1-score.

iv. Compare the three methods (kNN, distance-weighted kNN, fuzzy kNN). Discuss the differences you observe in their performance.

v. Analyze the effect of the parameter k. Explain how and why changing k influences the results.

vi. Analyze the effect of the fuzzifier m. Explain how and why changing m influences the behavior of fuzzy kNN.

vii. Report the confusion matrix for the best parameter setting (in terms of F1 score) for each of the 3 methods (i.e. 1 matrix for kNN, 1 for distance-weighted kNN, 1 for fuzzy kNN).

viii. Identify the class that is the most difficult to classify. Support your answer using the reported metrics and confusion matrices.

ix. Briefly interpret the confusion matrices. Which classes are most frequently confused with each other?

Report Requirements. In your report, clearly present the following:

- A brief description of your implementation. Explain how you implemented:
  - classical kNN,
  - distance-weighted kNN,
  - fuzzy kNN.

  Highlight any important details.
- All requested evaluation metrics and confusion matrices.
- Clear and concise answers to the analysis questions above.

Important Notes.

- Split the dataset into training and test sets using a **70%-30% split**. Use `random_state = 42` for reproducibility.
- **Normalize the features** by computing the mean and standard deviation on the training set only. Then apply the same transformation to both training and test sets. Do not use the test set when computing these values (to **avoid data leakage**).
- In all methods, use **Euclidean distance**.
- You are expected to implement nearest neighbor search efficiently. Instead of computing all distances and sorting the entire list, **maintain the k nearest neighbors using a data structure such as a heap (priority queue) of size k.**

  This allows you to keep track of the k closest points while scanning the training set, without storing or sorting all distances.

  Submissions that compute all pairwise distances and perform a full sort will still be accepted; however, they will receive a moderate penalty.